

PROVA DE APTIDÃO PROFISSIONAL

TRYPPIA

The logo for trypia, featuring the word "trypia" in a red, lowercase, sans-serif font.The logo for Agrupamento de Escolas Raul Proença, featuring a stylized blue and grey graphic above the text "Raul Proença" in blue.The logo for TGPSI, featuring a blue stylized brain icon above the text "TGPSI" in blue, with "CURSO PROFISSIONAL" in smaller blue text below.

CURSO PROFISSIONAL DE TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

NOME ALUNO: Rui Sousa

N.º: 19

PROFESSOR ORIENTADOR: Luís Fernandes

ANO LETIVO: 2025/2026

DEDICATÓRIA

Dedico este projeto ao Professor Luís Fernandes, pela dedicação, profissionalismo e apoio prestado ao longo do meu percurso académico, em especial nas aulas de Redes de Comunicação. Os conhecimentos que transmitiu, a disponibilidade para esclarecer dúvidas e a forma como incentivou os alunos a evoluírem foram essenciais para o meu crescimento, tanto a nível técnico como pessoal.

A sua orientação contribuiu para o desenvolvimento das competências que me permitiram realizar este projeto com maior confiança e autonomia. Ao longo do curso, demonstrou sempre empenho em ajudar os alunos a ultrapassar dificuldades, motivando-nos a procurar constantemente melhorar e a enfrentar novos desafios.

Por tudo o que me ensinou, pela paciência, pelo incentivo e pelo acompanhamento durante esta etapa importante da minha formação, deixo o meu sincero agradecimento. Esta dedicatória representa o reconhecimento pela influência positiva que teve no meu percurso académico e na concretização deste projeto.



AGRADECIMENTOS

Gostaria de expressar o meu sincero agradecimento a todas as pessoas que, de alguma forma, contribuíram para a realização deste projeto e para o meu percurso académico.

Em primeiro lugar, agradeço aos meus professores, em especial ao Professor Luís Fernandes, por todos os conhecimentos transmitidos, pela orientação, disponibilidade e incentivo ao longo do curso. O seu apoio foi essencial para o meu desenvolvimento e para a concretização deste projeto.

Agradeço também aos meus colegas de turma, pela entreatajuda, partilha de conhecimentos e espírito de colaboração demonstrados durante estes anos. O apoio mútuo contribuiu para tornar este percurso mais enriquecedor e motivador.

Um agradecimento muito especial à minha família, pelo apoio incondicional, compreensão, incentivo e confiança que sempre depositaram em mim. O seu encorajamento foi fundamental para superar os desafios encontrados ao longo desta etapa.

Por fim, agradeço a todos aqueles que, direta ou indiretamente, contribuíram para a minha formação pessoal e profissional e para a concretização deste projeto. O apoio e a motivação recebidos foram fundamentais para alcançar este objetivo.



ÍNDICE

Dedicatória.....	I
Agradecimentos	I
Índice	II
Índice de figuras	IV
Índice de tabelas	V
Lista de siglas e acrónimos	VI
1. Introdução.....	7
1.1. Fundamentação.....	8
1.2. Descrição do projeto	8
1.3. Objetivos do projeto	9
1.4. Estado do conhecimento	9
2. Planificação do projeto.....	11
2.1. Calendarização	11
2.1.1. Calendarização prevista	12
2.1.2. Calendarização realizada	13
2.2. Tecnologia utilizada.....	14
2.3. Protótipo	16
2.4. Base de Dados	16
3. Implementação do projeto	19
3.1 Desenvolvimento da página inicial.....	20
3.2 Secção de Hotéis	21
3.3 Secção De Experiências	21
3.4. Secção de voos e API Amadeus.....	22
3.5. Carrinho de compras	23
3.6. Checkout em etapas.....	24
3.7. Gestão de reservas.....	25
3.8. Perfil do utilizador	26
3.9. Área do vendedor	26
3.10. Backoffice administrativo	27



3.11. Explicação geral do código.....	28
3.12. Estrutura da Base de Dados.....	29
3.13. Sistema de Autenticação e Permissões.....	30
3.14. Organização das Rotas	31
3.15. Sistema de Upload de Imagens	32
3.18. Otimização da Aplicação	35
3.19. Segurança da Aplicação	36
4. Problemas e soluções encontradas	39
5. Conclusão	40
Referências bibliográficas.....	42



ÍNDICE DE FIGURAS

Figura 1 : Logo do Visual Studio Code	14
Figura 2 : Logo do Laravel	14
Figura 3 : Logo do php	15
Figura 4 : Logo do Tailwind CSS	15
Figura 5 : Diagrama BD	18
Figura 6 : Página inicial da plataforma	20
Figura 7 : Secção de hotéis disponíveis	21
Figura 8 : Secção de experiências 1	22
Figura 9 : Pesquisa de voos	23
Figura 10 : Carrinho de compras do Trypia	24
Figura 11 : Checkout	25
Figura 12 : Área das minhas reservas	25
Figura 13 : Perfil do utilizador	26
Figura 14 : Área do vendedor	27
Figura 15 : Dashboard administrativo	28
Figura 16 : Diagrama BD	29
Figura 17 : Código Página de autenticação	31
Figura 18 : Excerto do ficheiro routes/web	32
Figura 19 : Código inserir imagens	32
Figura 20 : Método Validate()	33
Figura 21 : Teste Backoffice	35
Figura 22 : Código Layout principal	36
Figura 23 : Token CSRF	38

ÍNDICE DE TABELAS

Tabela 1 : Cronograma inicial.....	12
Tabela 2 :Cronograma final.....	13



LISTA DE SIGLAS E ACRÓNIMOS

API - Application Programming Interface

BD - Base de Dados

CRUD - Create, Read, Update and Delete

CSS - Cascading Style Sheets

HTML - HyperText Markup Language

IDE - Integrated Development Environment

MVC - Model, View, Controller

PAP - Prova de Aptidão Profissional

PHP - Hypertext Preprocessor

SQL - Structured Query Language

TGPSI - Técnico de Gestão e Programação de Sistemas Informáticos

UI - User Interface

UX - User Experience

1. INTRODUÇÃO

A Prova de Aptidão Profissional (PAP) consiste na apresentação e defesa, perante um júri, de um projeto desenvolvido pelo aluno ao longo do seu percurso formativo. Este projeto permite demonstrar os conhecimentos, competências técnicas e capacidades de organização adquiridos durante o Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos, aplicando-os na resolução de um problema real através do desenvolvimento de uma solução informática.

A realização da PAP representa a etapa final da formação profissional, constituindo uma oportunidade para integrar conhecimentos de programação, bases de dados, desenvolvimento web, análise de requisitos, arquitetura de software e integração de serviços externos. Para além da componente técnica, este trabalho exige capacidade de planeamento, autonomia, investigação e resolução de problemas.

O presente relatório descreve todas as fases do desenvolvimento do projeto realizado no âmbito da PAP. Encontra-se organizado em cinco capítulos. O primeiro corresponde à introdução, onde é apresentado o enquadramento do projeto, a sua fundamentação, os objetivos e o estado do conhecimento. O segundo capítulo aborda a planificação do projeto e a calendarização das diferentes fases de desenvolvimento. O terceiro descreve a implementação técnica da aplicação, apresentando as principais funcionalidades desenvolvidas. O quarto capítulo identifica os principais problemas encontrados durante o desenvolvimento e as soluções adotadas. Por fim, o quinto capítulo apresenta as conclusões finais e uma reflexão sobre os conhecimentos adquiridos ao longo da realização do projeto.

O projeto desenvolvido tem o nome Trypia e consiste numa plataforma inteligente de gestão de viagens e reservas online. A aplicação foi concebida com o objetivo de reunir diferentes serviços turísticos numa única plataforma, permitindo aos utilizadores pesquisar hotéis, experiências e voos, adicionar serviços ao carrinho de compras e concluir reservas através de um processo de checkout organizado.

Ao longo do desenvolvimento foram aplicadas metodologias de trabalho que permitiram organizar as diferentes fases do projeto, desde a análise inicial dos requisitos até à implementação das funcionalidades finais. Foi dada especial importância à organização da base de dados, à qualidade da interface, à experiência do utilizador e à integração da API Amadeus para pesquisa de voos, procurando aproximar a aplicação ao funcionamento de plataformas profissionais existentes no mercado.

Este relatório descreve detalhadamente todo o processo de desenvolvimento do Trypia, apresentando as tecnologias utilizadas, a arquitetura da aplicação, as opções de implementação adotadas, os desafios encontrados durante o desenvolvimento e as soluções implementadas. No final

é apresentada uma apreciação crítica do trabalho realizado e dos conhecimentos técnicos adquiridos ao longo da execução da Prova de Aptidão Profissional.

1.1. FUNDAMENTAÇÃO

O turismo é uma área fortemente influenciada pela tecnologia. Plataformas como Booking, Airbnb, Expedia e outros serviços semelhantes alteraram a forma como os utilizadores pesquisam e reservam viagens. Em vez de recorrerem a vários contactos ou serviços separados, os utilizadores esperam atualmente encontrar informação organizada, imagens, preços, datas disponíveis e processos de reserva rápidos.

A ideia do Trypia nasceu da necessidade de criar uma plataforma que juntasse diferentes elementos de uma viagem: hotéis, experiências e voos. Esta centralização torna o processo mais simples, porque o utilizador deixa de navegar por várias plataformas e passa a encontrar os principais serviços num único website.

Outro fator determinante para a escolha deste projeto foi a possibilidade de consolidar e aplicar, de forma integrada, os conhecimentos adquiridos nas diferentes disciplinas do curso, promovendo a articulação entre conceitos teóricos e a sua aplicação prática.

Para além da componente técnica, o projeto permitiu trabalhar também a componente visual e funcional. A plataforma foi pensada para ter um aspeto moderno, simples e apelativo, semelhante ao estilo de aplicações reais utilizadas no mercado. Esta preocupação com a experiência do utilizador foi importante para tornar o projeto mais completo e profissional.

1.2. DESCRIÇÃO DO PROJETO

O Trypia é uma aplicação web desenvolvida em Laravel que permite gerir e reservar serviços relacionados com viagens. A plataforma possui três categorias principais: hotéis, experiências e voos. Os hotéis e experiências são registados como anúncios na base de dados, enquanto os voos podem ser importados através da API Amadeus e convertidos para o mesmo formato utilizado pelo restante sistema.

A aplicação permite que um utilizador navegue pelas categorias disponíveis, consulte a página detalhada de cada anúncio, visualize preço, descrição, localização, avaliação e imagens, adicione serviços ao carrinho e conclua o processo através de um checkout. Este checkout recolhe datas, dados pessoais e dados de pagamento simulado, criando no final uma reserva associada ao utilizador.

A plataforma inclui também diferentes tipos de utilizador. O visitante pode explorar o website e criar conta; o utilizador autenticado pode fazer reservas e consultar o histórico; o vendedor pode

gerir os seus anúncios; e o administrador possui acesso a um *backoffice* com gestão de utilizadores, anúncios e reservas.

Uma das funcionalidades mais relevantes é a integração dos voos no mesmo sistema de carrinho e checkout usado pelos hotéis e experiências. Esta solução permitiu unificar o processo de reserva, tornando o funcionamento da plataforma mais consistente e mais fácil de explicar.

1.3. OBJETIVOS DO PROJETO

O principal objetivo deste projeto consistiu no desenvolvimento de uma plataforma de viagens funcional, organizada e visualmente apelativa, capaz de simular uma aplicação real de reservas online.

Para atingir esse objetivo, foi desenvolvida uma interface moderna e responsiva, proporcionando uma navegação simples e intuitiva. Foi também implementado um sistema de autenticação de utilizadores e diferentes níveis de permissões, distinguindo utilizadores, vendedores e administradores. A plataforma permite pesquisar hotéis, experiências e voos, recorrendo à integração da API Amadeus para obtenção de informações atualizadas sobre voos.

Além disso, foi desenvolvido um carrinho de compras que permite reunir vários serviços antes da reserva final, bem como um processo de checkout organizado, onde o utilizador confirma todos os dados da reserva. Após a confirmação, a informação é armazenada na base de dados e fica disponível na área de reservas do utilizador.

O projeto inclui ainda uma área de perfil do utilizador, uma área destinada aos vendedores para a gestão dos seus anúncios e um painel administrativo para a gestão global da plataforma. A base de dados foi desenvolvida em MySQL, utilizando o MySQL Workbench para a sua modelação e gestão, permitindo armazenar toda a informação necessária ao funcionamento da aplicação. Por fim, pretendeu-se desenvolver uma aplicação completa, funcional e organizada, demonstrando os conhecimentos e competências adquiridos ao longo do curso e cumprindo os objetivos definidos para a Prova de Aptidão Profissional.

1.4. ESTADO DO CONHECIMENTO

Antes do início do desenvolvimento do projeto, foi necessário realizar um estudo sobre os principais conceitos relacionados com plataformas digitais de viagens e reservas online. Esta fase permitiu compreender o funcionamento das aplicações existentes no mercado, bem como identificar as funcionalidades consideradas essenciais para proporcionar uma boa experiência ao utilizador.

Foram analisadas plataformas como o Booking, Airbnb e Expedia, permitindo perceber a importância de funcionalidades como a pesquisa de alojamentos, apresentação organizada da informação, utilização de filtros, processos de reserva simples e interfaces intuitivas. Esta análise serviu de referência para definir a estrutura e as funcionalidades da plataforma Trypia.

Foi igualmente necessário aprofundar conhecimentos sobre desenvolvimento de aplicações web utilizando a *framework* Laravel e a arquitetura MVC, compreendendo a separação entre a lógica da aplicação, a gestão dos dados e a interface apresentada ao utilizador. Este estudo revelou-se fundamental para desenvolver uma aplicação organizada, de fácil manutenção e preparada para futuras melhorias.

Outro aspeto importante consistiu na compreensão do funcionamento das APIs, em particular da API Amadeus, utilizada para disponibilizar informações sobre voos em tempo real. Foi também necessário estudar o formato JSON, utilizado na troca de dados entre a aplicação e a API, bem como a forma de processar essa informação antes da sua apresentação ao utilizador.

Por fim, foi aprofundado o estudo sobre bases de dados relacionais em MySQL, incluindo a modelação das tabelas, a definição das relações entre elas e a organização da informação necessária ao funcionamento da plataforma. Estes conhecimentos permitiram desenvolver uma base de dados estruturada e adequada às necessidades do projeto, assegurando o correto armazenamento e gestão das reservas, utilizadores e restantes informações da aplicação.

2. PLANIFICAÇÃO DO PROJETO

A fase de planeamento foi essencial para organizar o desenvolvimento do projeto e definir uma estratégia de trabalho que permitisse atingir todos os objetivos estabelecidos. Antes do início da implementação, foi feita uma análise das funcionalidades que a plataforma deveria possuir e estabelecida uma ordem lógica para o seu desenvolvimento.

Inicialmente, foi definido o objetivo principal do projeto e identificados os requisitos necessários para a construção da aplicação. Posteriormente, foi elaborado um plano de trabalho dividido por várias fases, permitindo desenvolver cada funcionalidade de forma gradual e organizada, facilitando a identificação e correção de eventuais problemas ao longo do processo.

Durante o desenvolvimento, foi necessário adaptar o planeamento inicial sempre que surgiram novas necessidades ou dificuldades. Estas alterações permitiram melhorar a organização do projeto e garantir que todas as funcionalidades previstas fossem concluídas dentro do tempo disponível.

A existência de um planeamento estruturado contribuiu para uma melhor gestão do trabalho, permitindo acompanhar a evolução do projeto, cumprir os objetivos definidos e desenvolver uma aplicação organizada, funcional e preparada para a apresentação final da Prova de Aptidão Profissional.

2.1. CALENDARIZAÇÃO

Antes do início do desenvolvimento foi elaborada uma calendarização com o objetivo de organizar todas as fases do projeto e distribuir as tarefas ao longo do período disponível para a realização da Prova de Aptidão Profissional. Este planeamento permitiu definir uma sequência lógica de trabalho, facilitando o acompanhamento da evolução do projeto e garantindo que todas as etapas fossem concluídas dentro dos prazos estabelecidos.

A calendarização incluiu as principais fases do projeto, desde a análise e planeamento inicial até ao desenvolvimento, testes, correção de erros e preparação da apresentação final. Embora tenha sido necessário realizar alguns ajustes ao longo da execução do projeto devido a dificuldades encontradas e à implementação de novas funcionalidades, o cronograma serviu como uma ferramenta essencial para manter a organização e assegurar o cumprimento dos objetivos definidos.

2.1.1. CALENDARIZAÇÃO PREVISTA

A calendarização prevista foi elaborada com o objetivo de organizar todas as fases do desenvolvimento do projeto, distribuindo as diferentes tarefas ao longo do ano letivo. Este planeamento permitiu definir uma sequência lógica de trabalho, estabelecer prioridades e acompanhar a evolução do projeto desde a fase inicial até à sua conclusão, garantindo uma gestão mais eficiente do tempo disponível.

O cronograma apresentado inclui as principais etapas do desenvolvimento do Trypia, como a pesquisa sobre o tema, a análise de requisitos, a conceção da base de dados, o desenvolvimento das funcionalidades da aplicação, a realização de testes, a correção de erros e a preparação da documentação e da apresentação final da PAP. Embora tenham sido necessários alguns ajustes durante o desenvolvimento do projeto, esta calendarização serviu como referência para assegurar o cumprimento dos objetivos estabelecidos e dos prazos definidos.

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Debate de ideias com a turma									
Pesquisa sobre o tema									
Definição dos objetivos									
Desenvolvimento da aplicação									
Integração da API Amadeus									
Testes e correção de erros									
Desenvolvimento do backoffice									
Preparação da Apresentação da PAP									
Entrega e defesa da PAP									

Tabela 1 : Cronograma inicial

2.1.2. CALENDARIZAÇÃO REALIZADA

A calendarização realizada apresenta as diferentes etapas que foram efetivamente desenvolvidas ao longo da execução do projeto. Embora o planeamento inicial tenha servido de orientação, algumas tarefas sofreram ajustes devido às dificuldades encontradas durante o desenvolvimento e à necessidade de implementar novas funcionalidades. Desta forma, o cronograma final reflete o trabalho realmente realizado até à conclusão da aplicação.

Ao longo do projeto foram concluídas várias fases, começando pela modelação da base de dados e configuração inicial do Laravel, seguindo-se o desenvolvimento das diferentes áreas da plataforma, como a página inicial, hotéis, experiências, voos, carrinho de compras, checkout, área de utilizador, área de vendedor e painel de administração. Posteriormente foram realizados testes, corrigidos erros identificados durante o desenvolvimento, preparada a documentação do projeto e organizada a apresentação final da Prova de Aptidão Profissional. Este cronograma representa a evolução real do projeto e demonstra o cumprimento das principais etapas previstas para a sua concretização.

ETAPAS	OUT	NOV	DEZ	JAN	FEV	MAR	ABR	MAI	JUN
Modelação da base de dados									
Desenvolvimento da aplicação									
Integração da API Amadeus									
Desenvolvimento do carrinho de compras									
Implementação do checkout									
Desenvolvimento da área de vendedor									
Desenvolvimento da área de utilizador									
Desenvolvimento do painel de administração									
Testes e correção de erros									
Entrega e defesa da PAP									

Tabela 2 :Cronograma final

2.2. TECNOLOGIA UTILIZADA

O Visual Studio Code (VS Code) foi o ambiente de desenvolvimento utilizado durante todo o projeto. Trata-se de um editor de código desenvolvido pela Microsoft, bastante utilizado no desenvolvimento web devido à sua rapidez, simplicidade e elevado número de extensões disponíveis.

Durante o desenvolvimento do Trypia, o Visual Studio Code foi utilizado para escrever, editar e organizar todo o código da aplicação, incluindo ficheiros PHP, Blade, HTML, CSS, JavaScript e ficheiros de configuração do Laravel. Foram ainda utilizadas extensões para melhorar a produtividade, como realce de sintaxe, formatação automática do código e integração com o Git.

Esta ferramenta permitiu desenvolver a aplicação de forma organizada e facilitou a identificação e correção de erros ao longo do projeto.



Figura 1 : Logo do Visual Studio Code

O Laravel é uma framework de desenvolvimento web para PHP baseada na arquitetura MVC (Model-View-Controller). O seu principal objetivo é facilitar a criação de aplicações modernas, organizadas e seguras, disponibilizando várias funcionalidades já preparadas, como autenticação, gestão de rotas, acesso à base de dados, validação de formulários e sistema de templates.

No projeto Trypia, o Laravel constituiu a base de toda a aplicação. A framework foi utilizada para organizar o projeto em Models, Views e Controllers, permitindo separar a lógica da aplicação da interface gráfica. Também foi utilizada na criação das rotas, autenticação de utilizadores, gestão de permissões, comunicação com a base de dados, integração da API Amadeus e implementação do sistema de reservas.



Figura 2 : Logo do Laravel

A utilização do Laravel tornou o desenvolvimento mais rápido, organizado e facilitou a manutenção futura da aplicação.

O PHP (Hypertext Preprocessor) é uma linguagem de programação executada no servidor, amplamente utilizada no desenvolvimento de aplicações web dinâmicas. O código PHP é processado antes de a página ser enviada ao navegador, permitindo gerar conteúdos personalizados de acordo com os dados existentes na base de dados ou introduzidos pelo utilizador.

No projeto Trypia, o PHP foi utilizado para desenvolver toda a lógica da aplicação. Através desta linguagem foram implementadas funcionalidades como autenticação de utilizadores, pesquisa de hotéis e voos, gestão do carrinho de compras, processamento das reservas, comunicação com a API Amadeus e interação com a base de dados MySQL.

O PHP permitiu ainda criar uma aplicação dinâmica, respondendo aos pedidos dos utilizadores em tempo real.



Figura 3 : Logo do php

O Tailwind CSS é uma framework CSS baseada em classes utilitárias, permitindo construir interfaces modernas sem necessidade de escrever grandes quantidades de folhas de estilo personalizadas.

Ao contrário de outras frameworks, o Tailwind fornece pequenas classes que podem ser combinadas diretamente no código HTML ou Blade para definir cores, espaçamentos, tamanhos, margens, tipografia, posicionamento e comportamento responsivo.

No projeto Trypia, o Tailwind CSS foi utilizado para desenvolver toda a interface gráfica da plataforma, incluindo a página inicial, listagens de hotéis e experiências, formulários, carrinho de compras, checkout, perfil do utilizador, área do vendedor e painel administrativo.

A utilização desta framework permitiu obter uma interface moderna, consistente, responsiva e adaptada a diferentes dispositivos, reduzindo significativamente o tempo de desenvolvimento.



Figura 4 : Logo do Tailwind CSS

2.3. PROTÓTIPO

Os projetos de software apresentam os mockups da aplicação.

Os projetos de hardware apresentam os desenhos 3D das peças a imprimir indicando se são da sua autoria ou se são de terceiros.

(perdi a foto do mockup) não está no pre projeto)

2.4. BASE DE DADOS

A base de dados do projeto Trypia foi desenvolvida em MySQL Workbench, permitindo estruturar e relacionar toda a informação necessária ao funcionamento da plataforma. Foram utilizadas tabelas relacionadas através de chaves primárias (Primary Keys) e chaves estrangeiras (Foreign Keys), garantindo a integridade dos dados.

As principais tabelas da aplicação são:

Users – Armazena os dados dos utilizadores registados, incluindo nome, email, palavra-passe, função (utilizador, vendedor ou administrador) e restantes informações pessoais. Esta tabela é a base para o sistema de autenticação e permissões.

Listings – Contém todos os anúncios publicados na plataforma, incluindo hotéis, experiências e outros serviços turísticos. Cada anúncio está associado a um utilizador (vendedor) através do campo user_id.

Cart_Items – Guarda temporariamente os serviços adicionados ao carrinho de compras antes da confirmação da reserva. Cada registo está associado ao utilizador e ao anúncio selecionado.

Reservations – Armazena as reservas concluídas pelos utilizadores, incluindo datas, número de hóspedes, preço total e estado da reserva. Cada reserva está associada ao utilizador que efetuou a compra.

Flights – Guarda as informações relativas aos voos obtidos através da integração com a API Amadeus, permitindo apresentar e utilizar esses dados durante o processo de reserva.

Relações entre as tabelas

A tabela Users possui uma relação um-para-muitos com a tabela Listings, uma vez que um vendedor pode criar vários anúncios.

A tabela Listings possui igualmente uma relação um-para-muitos com a tabela Cart_Items, permitindo que o mesmo anúncio seja adicionado ao carrinho por diferentes utilizadores.

A tabela Users está também relacionada com Cart_Items e Reservations, permitindo identificar o proprietário do carrinho e o utilizador responsável por cada reserva efetuada.

Esta estrutura relacional permite organizar a informação de forma consistente, evitando duplicação de dados e facilitando a gestão das reservas, anúncios e utilizadores da plataforma.

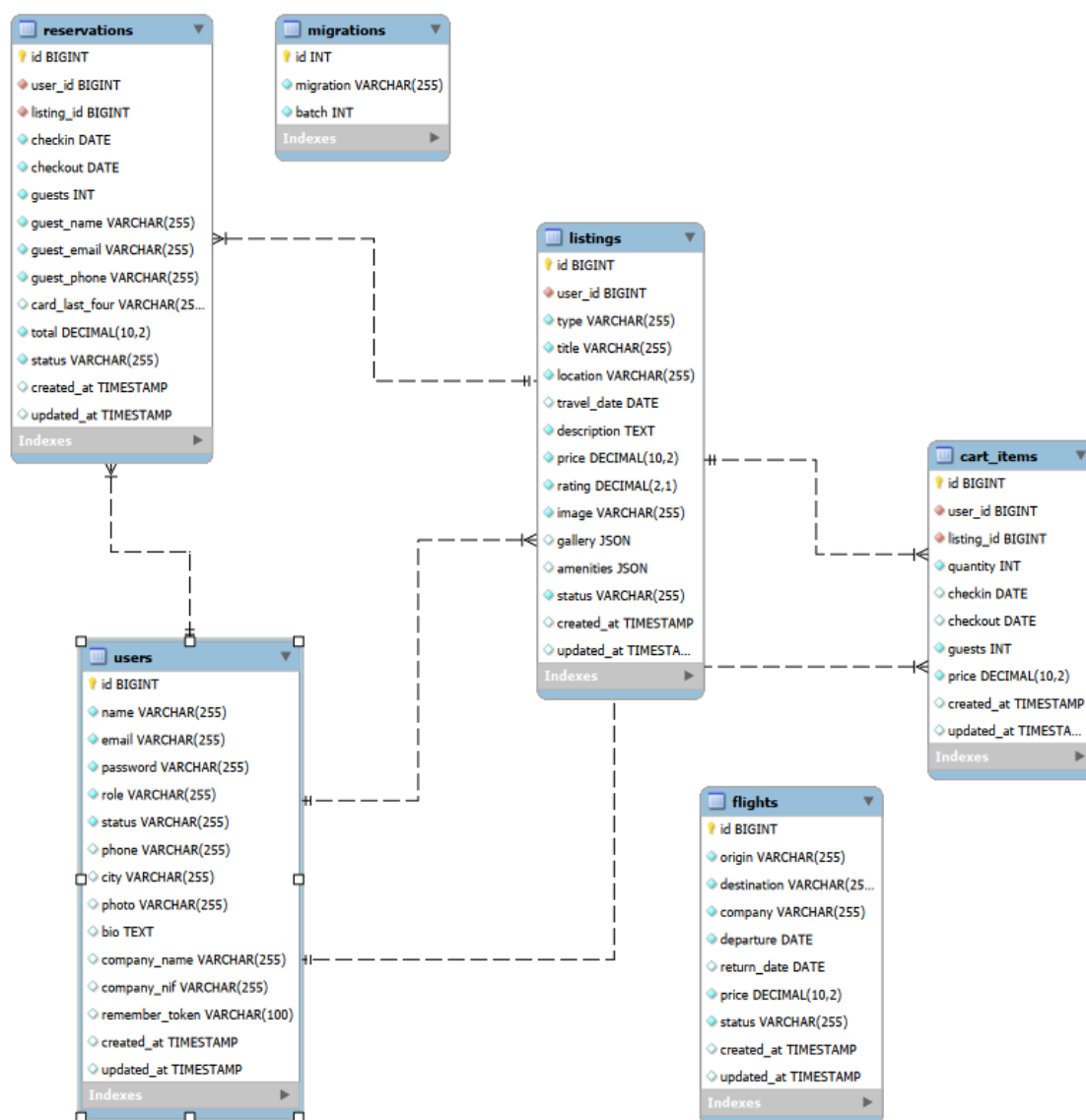


Figura 5 : Diagrama BD

3. IMPLEMENTAÇÃO DO PROJETO

A implementação do projeto Trypia foi realizada utilizando o framework Laravel, seguindo a arquitetura MVC (Model-View-Controller), permitindo separar a lógica da aplicação, a interface gráfica e o acesso à base de dados. Esta estrutura tornou o desenvolvimento mais organizado, facilitando a manutenção e a implementação de novas funcionalidades.

Numa primeira fase foi criado o projeto Laravel e configurado o ambiente de desenvolvimento no Laragon. De seguida, foi estabelecida a ligação à base de dados MySQL, criada e gerida através do MySQL Workbench, onde foram definidas as tabelas, relações e restrições necessárias ao funcionamento da aplicação.

Após a configuração inicial, foram desenvolvidos os Models, responsáveis pela comunicação com a base de dados, os Controllers, onde foi implementada toda a lógica da aplicação, e as Views, responsáveis pela apresentação da interface ao utilizador através do motor de templates Blade. Para a estilização das páginas foi utilizado Tailwind CSS, permitindo criar uma interface moderna, responsiva e intuitiva.

Foi implementado um sistema completo de autenticação de utilizadores, permitindo diferentes níveis de acesso para utilizadores, vendedores e administradores. Cada perfil possui permissões específicas, garantindo que apenas pode aceder às funcionalidades correspondentes ao seu papel dentro da plataforma.

A base de dados foi estruturada para armazenar informações relativas aos utilizadores, anúncios, hotéis, experiências, voos, carrinho de compras e reservas. Foram criadas relações entre as tabelas através de chaves estrangeiras, assegurando a integridade e consistência dos dados armazenados.

Para a pesquisa de voos foi integrada a API Amadeus. A aplicação envia pedidos para a API com os parâmetros introduzidos pelo utilizador, como origem, destino e datas da viagem. As respostas são recebidas em formato JSON, processadas pelos controllers e posteriormente apresentadas na interface da aplicação de forma organizada.

Foi igualmente desenvolvido um carrinho de compras, permitindo ao utilizador adicionar diferentes serviços turísticos antes de concluir a reserva. No processo de checkout, todos os dados são validados e, após confirmação, a reserva é registada na base de dados, ficando disponível na área pessoal do utilizador.

Durante todo o desenvolvimento foram realizados testes contínuos para validar o correto funcionamento de cada funcionalidade, corrigindo erros sempre que necessário e efetuando melhorias tanto na estrutura do código como na experiência de utilização da plataforma. Este processo permitiu obter uma aplicação estável, organizada e preparada para utilização.

3.1 DESENVOLVIMENTO DA PÁGINA INICIAL

A página inicial foi desenvolvida para servir como ponto de entrada da plataforma, permitindo ao utilizador aceder rapidamente às principais funcionalidades disponíveis. Nesta página são apresentados os serviços de hotéis, experiências e voos, bem como uma barra de pesquisa que facilita a navegação e a procura de conteúdos.

A construção desta página seguiu a arquitetura MVC (Model-View-Controller) utilizada pelo Laravel. A interface visível ao utilizador foi desenvolvida através de uma View, utilizando o motor de templates Blade, responsável pela apresentação do conteúdo e pela organização dos diferentes elementos gráficos da página.

O Controller associado à página inicial é responsável por receber o pedido do utilizador, obter a informação necessária e enviá-la para a View. Toda a lógica necessária para carregar os hotéis populares, experiências em destaque e restantes conteúdos apresentados na página é executada neste componente.

Os Models permitem comunicar com a base de dados, representando as diferentes tabelas utilizadas pela aplicação, como utilizadores, anúncios, hotéis, experiências e reservas. Através destes modelos é possível consultar, inserir, atualizar e eliminar informação de forma organizada, recorrendo ao ORM Eloquent disponibilizado pelo Laravel.

Para criar e atualizar a estrutura da base de dados foram utilizadas Migrations, que definem as tabelas, colunas, tipos de dados e relações entre elas. Desta forma, toda a estrutura da base de dados pode ser criada automaticamente e mantida de forma consistente durante o desenvolvimento da aplicação.

Por fim, a página foi desenvolvida com um design responsivo utilizando Tailwind CSS, garantindo uma boa experiência de utilização em diferentes dispositivos, como computadores, tablets e smartphones.

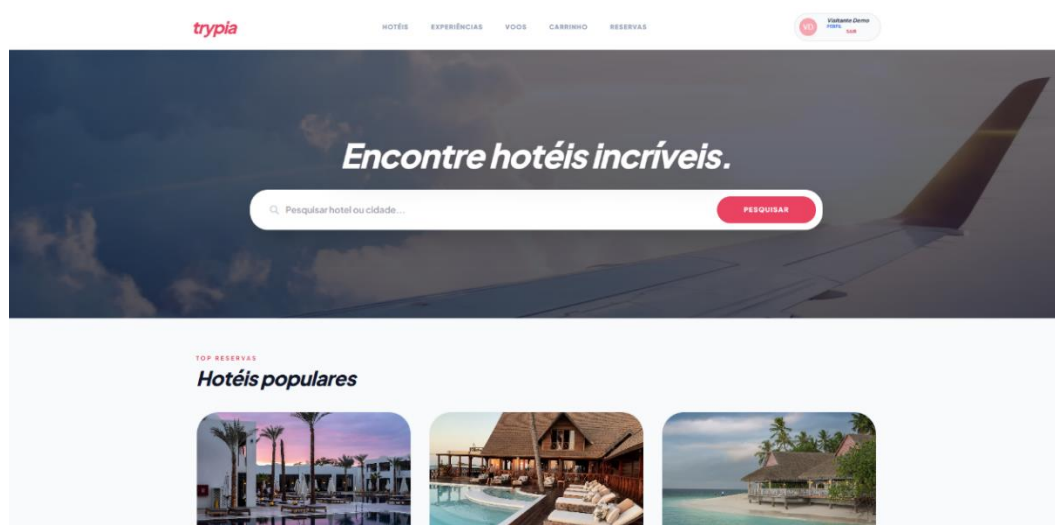


Figura 6 : Página inicial da plataforma

3.2 SECÇÃO DE HOTÉIS

A secção de hotéis apresenta os alojamentos disponíveis para reserva. Cada hotel é apresentado através de um cartão com imagem, nome, localização, avaliação e preço. O utilizador pode abrir a página de detalhes do hotel, consultar informações adicionais e iniciar o processo de reserva ou adicionar o serviço ao carrinho de compras.

Os dados apresentados nesta página são obtidos diretamente da base de dados, onde cada hotel possui informação como descrição, localização, preço, imagens e disponibilidade. Durante o carregamento da página, a aplicação consulta estes registos e apresenta-os automaticamente sob a forma de cartões organizados.

Foi ainda implementado um sistema de pesquisa e filtragem que permite ao utilizador encontrar alojamentos de acordo com os critérios selecionados. Esta funcionalidade melhora a experiência de utilização, tornando a pesquisa mais rápida e intuitiva.

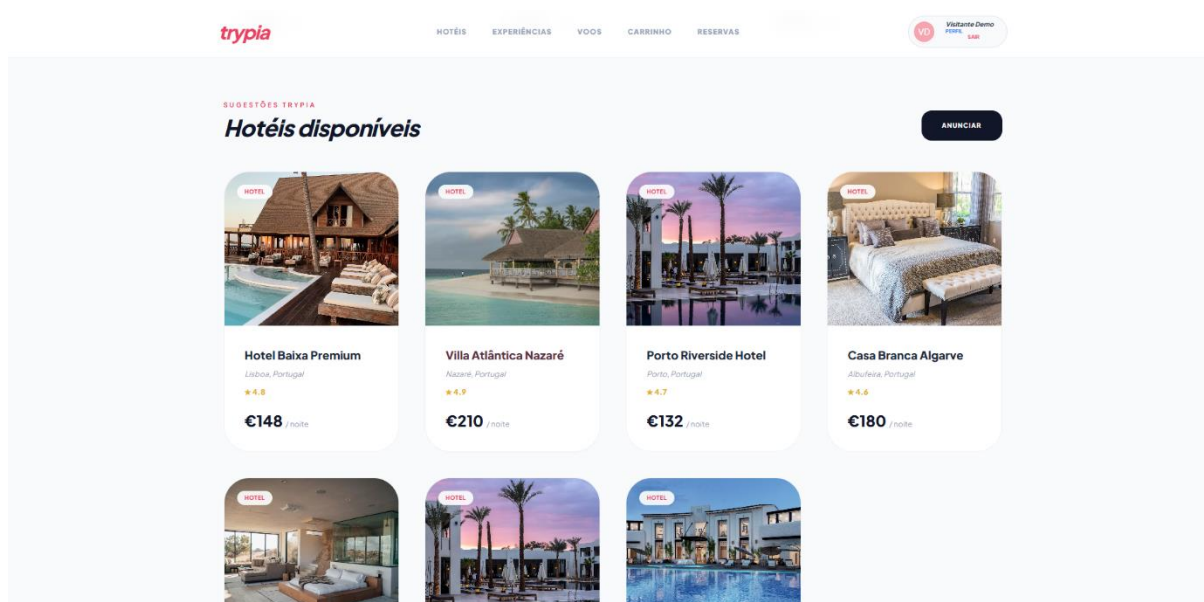


Figura 7 : Secção de hotéis disponíveis

3.3 SECÇÃO DE EXPERIÊNCIAS

A secção de experiências apresenta diversas atividades turísticas disponíveis para reserva, como visitas guiadas, passeios e outras experiências de lazer. Tal como na secção de hotéis, cada experiência é apresentada através de um cartão com imagem, nome, localização, preço e uma breve descrição.

Os dados das experiências são armazenados na base de dados e carregados automaticamente pela aplicação. O utilizador pode consultar os detalhes de cada experiência e adicioná-la ao carrinho de compras, permitindo combinar diferentes serviços numa única reserva.

Esta abordagem permitiu manter uma estrutura consistente em toda a plataforma, proporcionando uma navegação intuitiva e uma experiência de utilização uniforme entre as diferentes áreas do sistema.

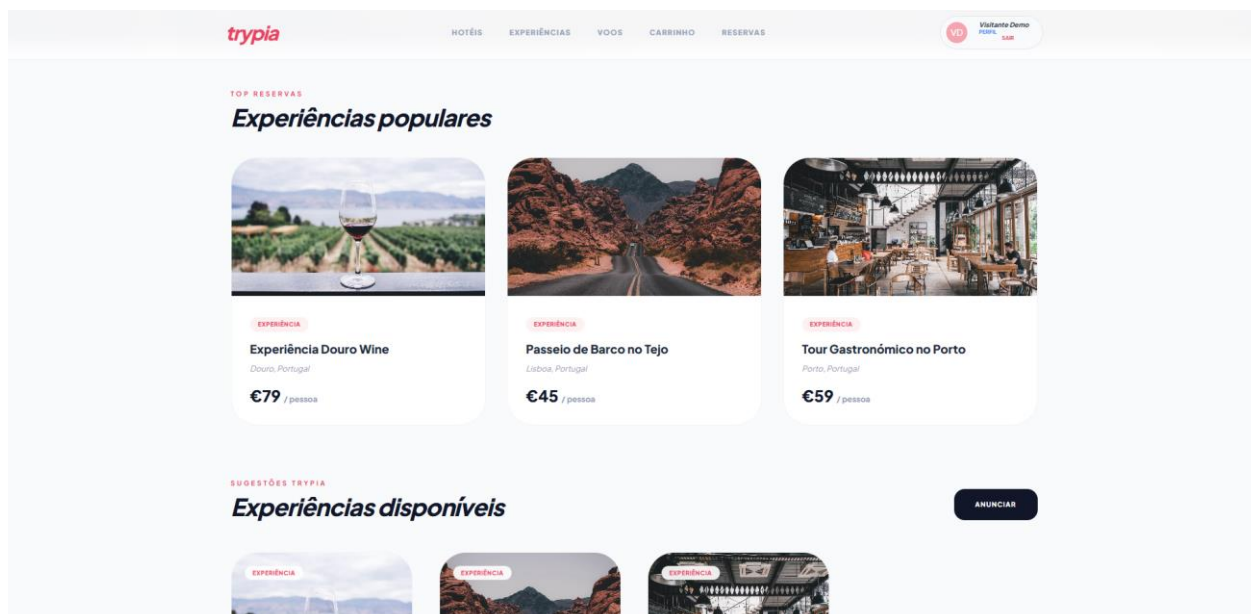


Figura 8 : Secção de experiências 1

3.4. SECÇÃO DE VOOS E API AMADEUS

A secção de voos foi integrada com a API Amadeus. O utilizador selecciona origem, destino e data; a aplicação envia o pedido à API e recebe ofertas de voos. Os resultados são convertidos em anúncios do tipo Voo, permitindo que sejam abertos, adicionados ao carrinho e reservados no mesmo sistema dos restantes serviços.

A integração da API exigiu a criação de um serviço próprio, responsável por obter o token de autenticação, enviar pedidos HTTP e tratar as respostas em formato JSON. Depois de recebidas, as ofertas de voos são normalizadas para a estrutura utilizada pelos anúncios da plataforma. Esta solução permitiu reutilizar o carrinho e o checkout já existentes.

Para facilitar a demonstração na PAP, a pesquisa de voos pode ser limitada a rotas estáveis, como Lisboa para Madrid, Paris, Londres ou Roma. Esta estratégia reduz a probabilidade de a API devolver resultados vazios e torna a apresentação mais controlada.

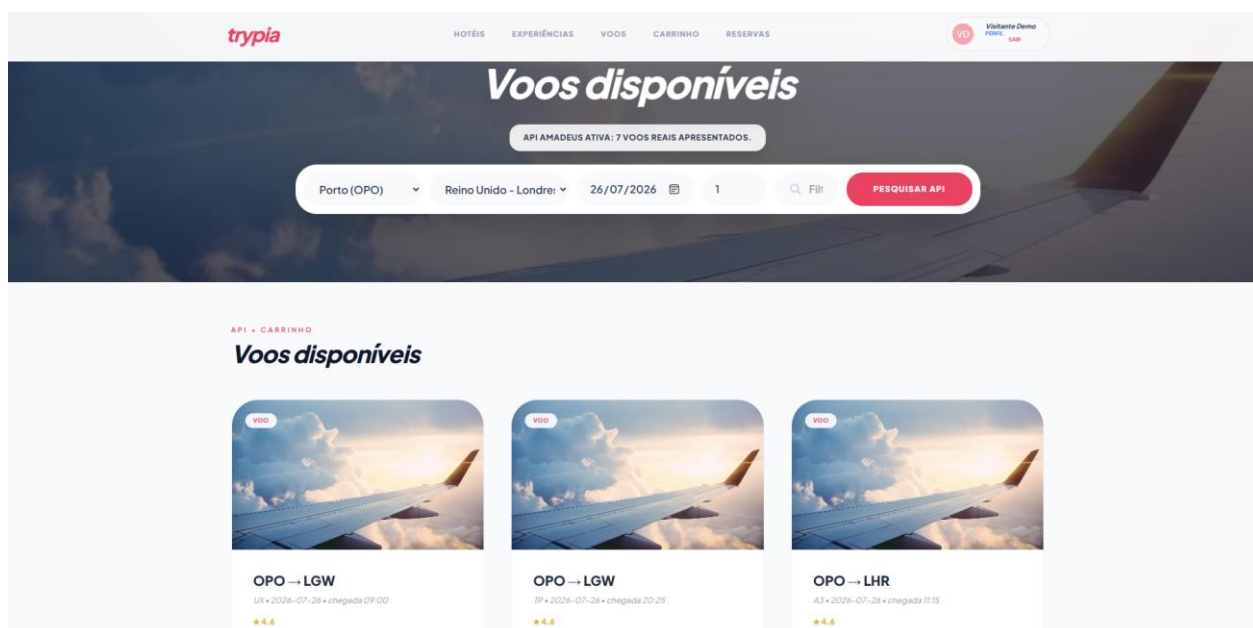


Figura 9 : Pesquisa de voos

3.5. CARRINHO DE COMPRAS

O carrinho de compras funciona como uma área temporária onde o utilizador pode juntar vários serviços antes de finalizar a reserva. Esta funcionalidade permite adicionar hotéis, experiências e voos no mesmo processo, aproximando o Trypia de uma plataforma real de viagens.

A tabela `cart_items` guarda os itens enquanto o utilizador ainda não confirmou a reserva. Esta tabela contém o utilizador, o anúncio, o preço, as datas e o número de pessoas. Quando o pagamento é confirmado, os itens do carrinho são transformados em reservas e o carrinho é limpo.

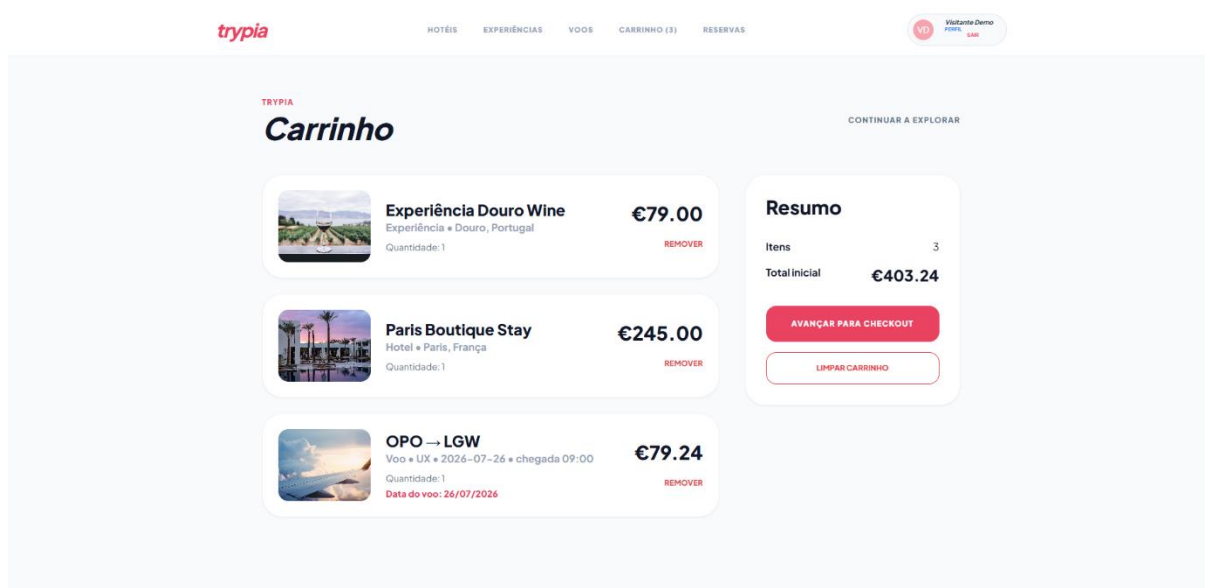
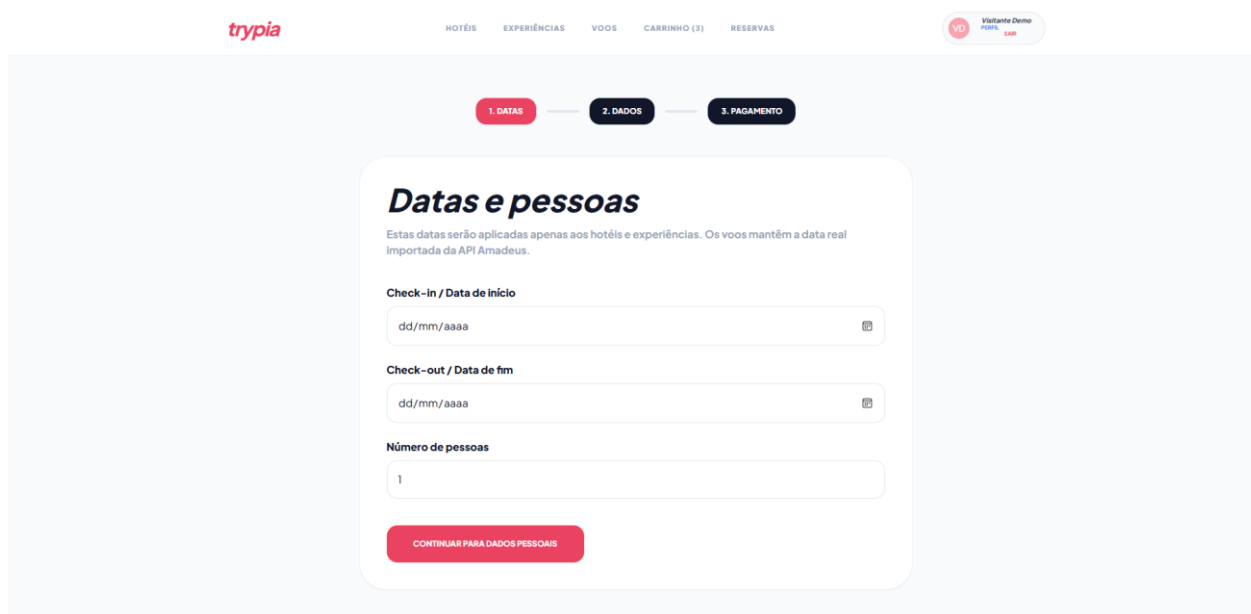


Figura 10: Carrinho de compras do Trypia

3.6. CHECKOUT EM ETAPAS

O checkout foi dividido em três etapas: escolha de datas e pessoas, preenchimento dos dados pessoais e pagamento simulado. Esta divisão torna o processo mais claro e evita que o utilizador tenha de preencher demasiada informação numa única página.

No caso dos voos, a data real importada da API é preservada. Já nos hotéis e experiências, as datas são escolhidas pelo utilizador durante o checkout. Esta diferença foi necessária porque um voo tem data própria, enquanto um hotel depende do check-in e check-out escolhidos.



trypia HOTÉIS EXPERIÊNCIAS VOOS CARRINHO (3) RESERVAS Visitante Demo

1. DATAS 2. DADOS 3. PAGAMENTO

Datas e pessoas

Estas datas serão aplicadas apenas aos hotéis e experiências. Os voos mantêm a data real importada da API Amadeus.

Check-in / Data de início
dd/mm/aaaa

Check-out / Data de fim
dd/mm/aaaa

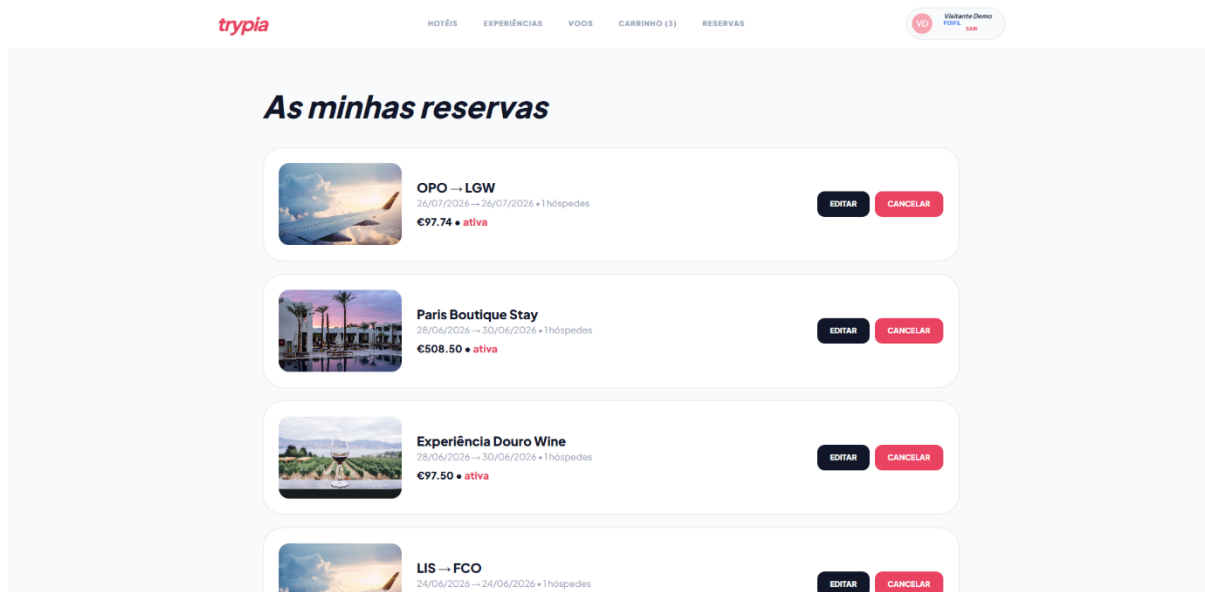
Número de pessoas
1

CONTINUAR PARA DADOS PESSOAIS

Figura 11 : Checkout

3.7. GESTÃO DE RESERVAS

Após a confirmação do checkout, são criados registos na tabela reservations. O utilizador pode consultar as suas reservas, visualizar os serviços reservados e cancelar ou editar informações, conforme as regras implementadas no sistema.



trypia HOTÉIS EXPERIÊNCIAS VOOS CARRINHO (3) RESERVAS Visitante Demo

As minhas reservas

	OPO → LGW 26/01/2026 → 26/07/2026 • 1 hóspede €97.74 • ativa	EDITAR CANCELAR
	Paris Boutique Stay 28/06/2026 → 30/06/2026 • 1 hóspede €508.50 • ativa	EDITAR CANCELAR
	Experiência Douro Wine 28/06/2026 → 30/06/2026 • 1 hóspede €97.50 • ativa	EDITAR CANCELAR
	LIS → FCO 24/06/2026 → 24/06/2026 • 1 hóspede €97.74 • ativa	EDITAR CANCELAR

Figura 12 : Área das minhas reservas

3.8. PERFIL DO UTILIZADOR

A área de perfil permite ao utilizador consultar e atualizar os seus dados. Esta funcionalidade é importante para dar maior controlo ao utilizador e tornar a aplicação mais completa.

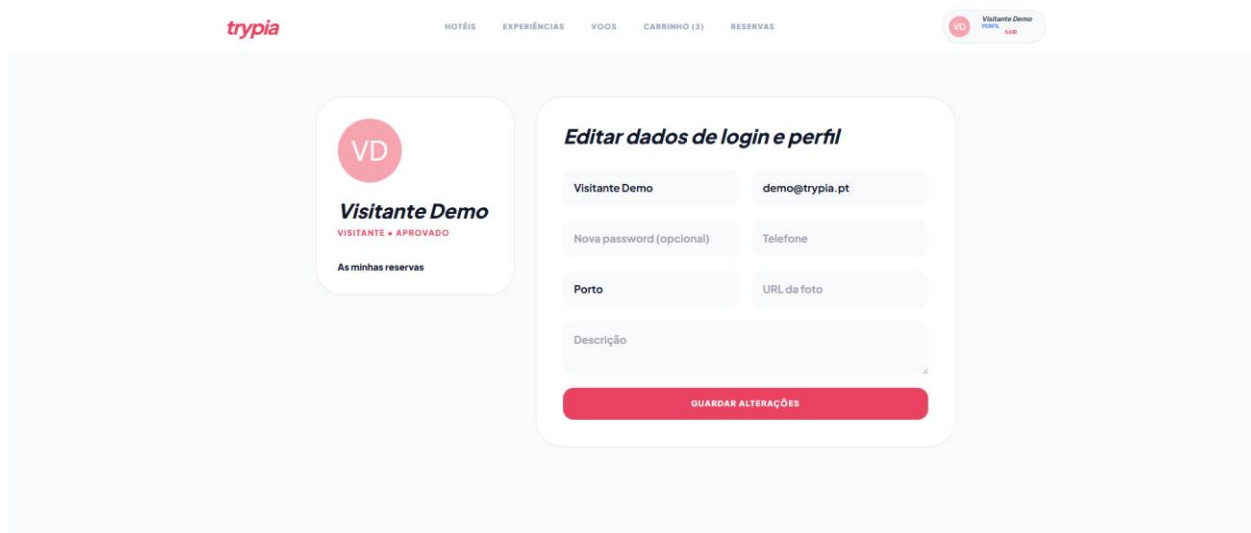


Figura 13 : Perfil do utilizador

3.9. ÁREA DO VENDEDOR

A área do vendedor permite criar, editar, pausar e eliminar anúncios. Esta funcionalidade transforma a plataforma num sistema com gestão de conteúdos, onde diferentes utilizadores podem contribuir com novos serviços.

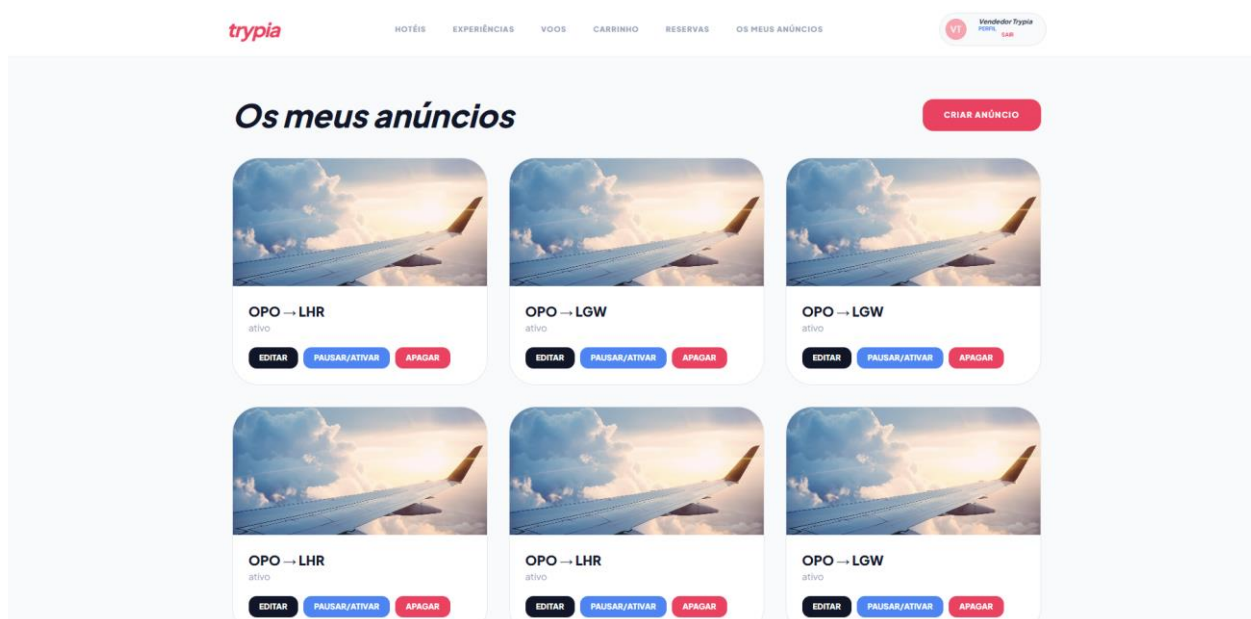


Figura 14 : Área do vendedor

3.10. BACKOFFICE ADMINISTRATIVO

O *backoffice* é a área reservada ao administrador. Permite gerir utilizadores, aprovar vendedores, controlar anúncios e consultar reservas. Esta área também apresenta estatísticas e gráficos, tornando a gestão da plataforma mais profissional.

O painel administrativo permite ao responsável pela plataforma acompanhar o funcionamento geral do sistema. A existência desta área demonstra o uso de permissões e separação de responsabilidades dentro da aplicação.

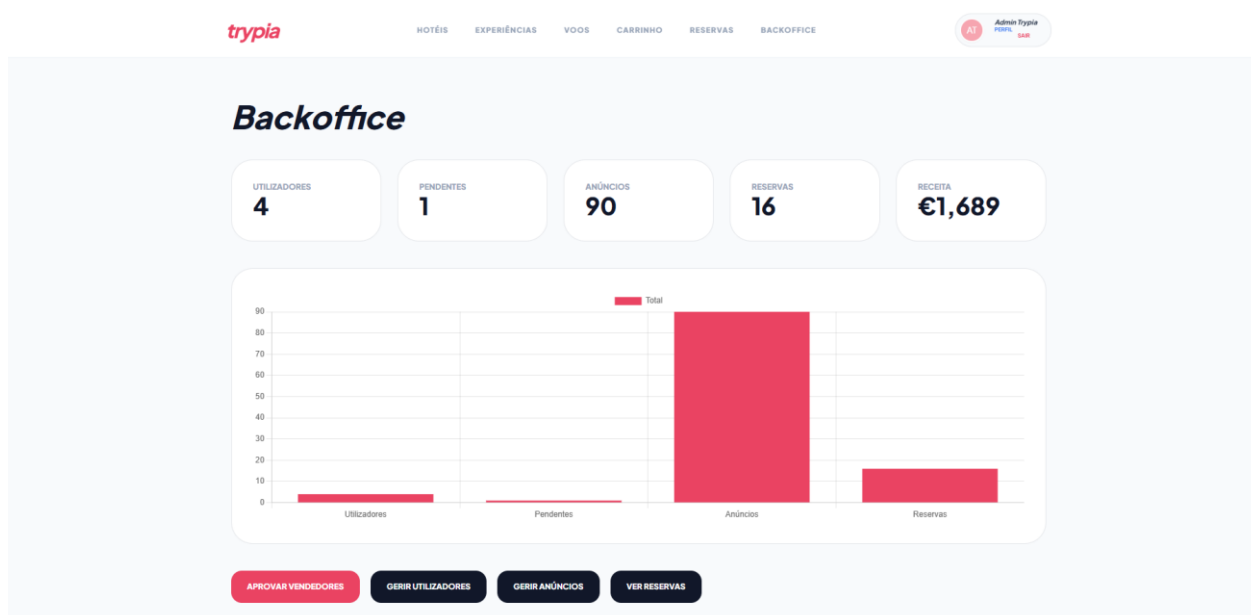


Figura 15 : Dashboard administrativo

3.11. EXPLICAÇÃO GERAL DO CÓDIGO

A aplicação foi desenvolvida recorrendo ao framework Laravel, que segue a arquitetura MVC. Esta arquitetura permite separar a lógica da aplicação, a apresentação da informação e o acesso à base de dados, tornando o código mais organizado, reutilizável e fácil de manter.

As rotas, definidas no ficheiro routes/web.php, são responsáveis por encaminhar os pedidos efetuados pelo utilizador para o respetivo controlador. Cada rota corresponde a uma funcionalidade específica da aplicação, como a página inicial, a pesquisa de hotéis, experiências e voos, o carrinho de compras, o checkout, a área do vendedor ou o painel de administração.

Os controladores são responsáveis por executar a lógica da aplicação. Recebem os pedidos enviados pelas rotas, processam os dados necessários, comunicam com os modelos e devolvem a informação às vistas. No projeto foram criados vários controladores, cada um responsável por um conjunto de funcionalidades específicas, permitindo uma melhor organização do código.

Os modelos representam as tabelas da base de dados e utilizam o ORM Eloquent, disponibilizado pelo Laravel. Através dos modelos é possível consultar, inserir, atualizar e eliminar informação sem recorrer diretamente a comandos SQL. Cada modelo corresponde a uma entidade da aplicação, como utilizadores, anúncios, hotéis, experiências, voos, reservas ou itens do carrinho.

As vistas, desenvolvidas com o motor de templates Blade, são responsáveis pela apresentação da informação ao utilizador. Nelas são construídas todas as páginas da aplicação, incluindo a página

inicial, listagens de hotéis e experiências, detalhes dos anúncios, carrinho de compras, checkout, área de reservas, área de vendedor e painel administrativo.

A estrutura da base de dados foi criada através das migrations do Laravel, permitindo definir tabelas, colunas, tipos de dados, índices e relações entre as diferentes entidades da aplicação. Este mecanismo facilita a criação e atualização da base de dados durante o desenvolvimento e garante que toda a estrutura pode ser recriada automaticamente em qualquer ambiente.

Para a gestão da base de dados foi utilizado o MySQL, sendo a modelação realizada através do MySQL Workbench. As relações entre tabelas foram implementadas recorrendo a chaves estrangeiras, garantindo a integridade referencial e evitando inconsistências nos dados armazenados.

3.12. ESTRUTURA DA BASE DE DADOS

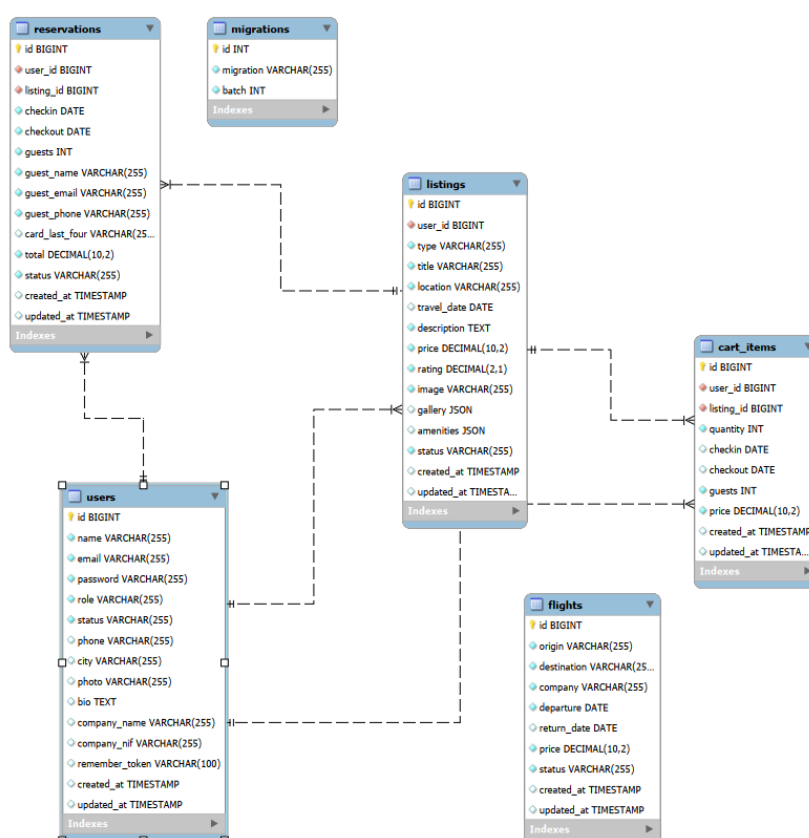


Figura 16 : Diagrama BD

A Figura 15 apresenta o Diagrama Entidade-Relacionamento da base de dados desenvolvida para o projeto Trypia. A estrutura foi modelada no MySQL Workbench e implementada na aplicação através de migrations do Laravel. As principais tabelas da aplicação são users, listings, reservations, cart_items e flights, estando relacionadas entre si através de chaves primárias e chaves estrangeiras. Esta organização permite armazenar e gerir de forma consistente toda a informação relativa aos utilizadores, anúncios, reservas, voos e carrinho de compras, garantindo a integridade dos dados e o correto funcionamento da plataforma.

3.13. SISTEMA DE AUTENTICAÇÃO E PERMISSÕES

O sistema de autenticação do Trypia foi desenvolvido utilizando as funcionalidades disponibilizadas pelo framework Laravel, garantindo um processo seguro de registo e início de sessão. Os utilizadores podem criar uma conta, autenticar-se na plataforma e aceder às funcionalidades disponíveis de acordo com o seu perfil. As palavras-passe são armazenadas de forma encriptada através do algoritmo bcrypt, aumentando a segurança dos dados dos utilizadores.

Após a autenticação, o Laravel cria uma sessão que identifica o utilizador durante toda a navegação. As áreas privadas da aplicação encontram-se protegidas através de middleware, impedindo o acesso por utilizadores não autenticados. Sempre que um utilizador tenta aceder a uma página reservada sem iniciar sessão, é automaticamente redirecionado para a página de login.

O sistema implementa ainda diferentes níveis de permissões, permitindo distinguir três tipos de utilizadores: utilizador, vendedor e administrador. Cada perfil possui acesso apenas às funcionalidades que lhe são destinadas. O utilizador pode efetuar reservas e consultar o seu histórico, o vendedor pode gerir os seus próprios anúncios e reservas, enquanto o administrador possui acesso total ao backoffice para gerir utilizadores, anúncios e o funcionamento geral da plataforma.

Esta separação de permissões foi implementada através do controlo de papéis (roles), verificados durante o acesso às diferentes rotas da aplicação. Desta forma, foi possível garantir uma maior segurança, organização e controlo sobre as funcionalidades disponibilizadas a cada tipo de utilizador.

```

1 reference | 0 overrides
public function login(Request $request)
{
    $data = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required']
    ]);

    $user = User::where('email', $data['email'])->first();

    if (!$user || !Hash::check($data['password'], $user->password)) {
        return back()->with('error', 'Email ou password incorretos.')->withInput();
    }

    session(['user_id' => $user->id]);
    return redirect()->route('home')->with('success', 'Login efetuado com sucesso.');
```

Figura 17: Código Página de autenticação

3.14. ORGANIZAÇÃO DAS ROTAS

No projeto Trypia, todas as rotas da aplicação foram definidas no ficheiro routes/web.php, responsável por gerir a navegação entre as diferentes páginas da plataforma. Cada rota associa um determinado endereço (URL) a um método existente num Controller, permitindo encaminhar corretamente os pedidos efetuados pelos utilizadores.

As rotas foram organizadas de acordo com as funcionalidades da aplicação. Existem rotas destinadas à página inicial, hotéis, experiências, voos, autenticação, perfil do utilizador, carrinho de compras, checkout, reservas, área do vendedor e painel administrativo. Esta organização permitiu manter o código estruturado e facilitar a manutenção da aplicação durante o desenvolvimento.

Sempre que um utilizador acede a uma determinada página, o Laravel identifica a rota correspondente e encaminha automaticamente o pedido para o respetivo Controller, onde é executada toda a lógica necessária. Posteriormente, o Controller devolve a View apropriada, apresentando a informação ao utilizador.

Embora as rotas estejam definidas num único ficheiro, o acesso às áreas privadas da plataforma é controlado pelos Controllers e pelo sistema de autenticação implementado na aplicação. Desta forma, apenas utilizadores autorizados podem aceder ao perfil, gerir anúncios, consultar reservas ou utilizar as funcionalidades administrativas.

A Figura seguinte apresenta um excerto do ficheiro routes/web.php, onde é possível observar a organização das principais rotas utilizadas no desenvolvimento do projeto Trypia.

```
Route::get('/meus-anuncios', [ListingController::class, 'mine'])->name('seller.listings');
Route::get('/meus-anuncios/criar', [ListingController::class, 'create'])->name('seller.listings.create');
Route::post('/meus-anuncios', [ListingController::class, 'store'])->name('seller.listings.store');
Route::get('/meus-anuncios/{listing}/editar', [ListingController::class, 'edit'])->name('seller.listings.edit');
Route::post('/meus-anuncios/{listing}', [ListingController::class, 'update'])->name('seller.listings.update');
Route::post('/meus-anuncios/{listing}/pausar', [ListingController::class, 'toggle'])->name('seller.listings.toggle');
Route::post('/meus-anuncios/{listing}/apagar', [ListingController::class, 'destroy'])->name('seller.listings.destroy');
```

Figura 18 : Excerto do ficheiro routes/web

3.15. SISTEMA DE UPLOAD DE IMAGENS

No projeto Trypia, as imagens dos anúncios são geridas através da utilização de URLs, não sendo necessário efetuar o upload de ficheiros para o servidor. Durante a criação ou edição de um anúncio, o vendedor introduz o endereço da imagem no respetivo formulário, sendo este posteriormente validado pelo Laravel antes de ser armazenado na base de dados.

A validação garante que o campo da imagem é obrigatório e que o valor introduzido corresponde a um URL válido. Após esta verificação, o endereço da imagem é associado ao anúncio juntamente com os restantes dados, como o título, localização, descrição, preço e classificação.

Sempre que um utilizador consulta um anúncio, a aplicação recupera automaticamente o URL armazenado na base de dados e apresenta a imagem correspondente na interface da plataforma. Este processo permite manter a aplicação mais simples, reduzindo a necessidade de armazenamento de ficheiros no servidor e facilitando a gestão das imagens.

Além da imagem principal, a aplicação permite também associar uma galeria de imagens através de vários links, proporcionando uma apresentação mais completa dos anúncios. Caso não sejam fornecidas imagens adicionais, o sistema utiliza automaticamente a imagem principal como elemento inicial da galeria.

A utilização de URLs revelou-se uma solução simples, eficiente e adequada aos objetivos do projeto, permitindo apresentar imagens de elevada qualidade sem aumentar a complexidade da aplicação nem o espaço de armazenamento utilizado.

```
public function update(Request $request, Listing $listing)
{
    $user = CurrentUser::requireSeller();
    abort_if($listing->user_id != $user->id, 403);
    $data = $this->validated($request);
    $data['gallery'] = $this->linesToArray($request->gallery_text) ?: [$data['image']];
    $data['amenities'] = $this->linesToArray($request->amenities_text);
    $listing->update($data);
    return redirect()->route('seller.listings')->with('success', 'Anúncio atualizado.');
```

Figura 19 : Código inserir imagens

3.16. VALIDAÇÃO DE DADOS

Durante o desenvolvimento do projeto Trypia, foi implementado um sistema de validação de dados com o objetivo de garantir que todas as informações introduzidas pelos utilizadores são corretas antes de serem armazenadas na base de dados. Esta validação contribui para a integridade dos dados, evita erros de funcionamento e melhora a segurança da aplicação.

O Laravel disponibiliza um sistema de validação integrado, que foi utilizado em todos os formulários da plataforma. Sempre que um utilizador submete um formulário, os dados são verificados antes de serem processados. Caso algum campo não cumpra as regras definidas, a operação é interrompida e são apresentadas mensagens de erro, permitindo ao utilizador corrigir a informação introduzida.

Entre as principais validações implementadas encontram-se a verificação dos campos obrigatórios, garantindo que informações essenciais, como o título, descrição, localização e preço dos anúncios, são sempre preenchidas. Foram também aplicadas validações ao preço, assegurando que apenas são aceites valores numéricos positivos, e à classificação (rating), limitando os valores ao intervalo permitido pela aplicação.

No sistema de gestão de imagens, o campo da imagem é validado para garantir que o valor introduzido corresponde a um URL válido, uma vez que a plataforma utiliza ligações externas para apresentar as imagens dos anúncios.

Durante o processo de reservas e checkout foram igualmente implementadas validações para as datas de entrada e saída, impedindo a seleção de datas inválidas ou inconsistentes. Nos formulários relacionados com reservas são ainda verificados os dados dos hóspedes, incluindo o número de passageiros e outras informações necessárias para concluir corretamente a reserva.

A utilização destas validações permitiu reduzir significativamente a ocorrência de erros, melhorar a consistência dos dados armazenados e proporcionar uma experiência de utilização mais segura e fiável.

```
private function validated(Request $request): array
{
    return $request->validate([
        'type' => ['required', 'in:Hotel,Experiência,Voo'],
        'title' => ['required', 'string', 'max:180'],
        'location' => ['required', 'string', 'max:180'],
        'description' => ['required', 'string'],
        'price' => ['required', 'numeric', 'min:1'],
        'rating' => ['required', 'numeric', 'min:0', 'max:5'],
        'image' => ['required', 'url']
    ]);
}
```

Figura 20 : Método Validate()

3.17. TESTES E CORREÇÃO DE ERROS

Ao longo do desenvolvimento da plataforma Trypia, foram realizados diversos testes com o objetivo de verificar o correto funcionamento de todas as funcionalidades implementadas. Estes testes permitiram identificar erros de programação, validar o comportamento da aplicação em diferentes situações e garantir uma experiência de utilização estável e segura.

Uma das primeiras funcionalidades testadas foi o sistema de autenticação, verificando o registo de novos utilizadores, o início de sessão e o encerramento da sessão. Foram igualmente testados os diferentes níveis de acesso da plataforma, garantindo que cada utilizador apenas podia aceder às funcionalidades correspondentes ao seu perfil (utilizador, vendedor ou administrador).

Na área do frontoffice, foram realizados testes às pesquisas de hotéis, experiências e voos, confirmando que os resultados apresentados correspondiam aos critérios introduzidos pelos utilizadores. No caso dos voos, foi também testada a comunicação com a API Amadeus, verificando a obtenção do token de autenticação, o envio dos pedidos e o tratamento das respostas recebidas em formato JSON.

O carrinho de compras foi igualmente sujeito a vários testes, incluindo a adição e remoção de serviços, atualização dos dados da reserva e cálculo automático do valor total. Posteriormente, foi testado todo o processo de checkout, verificando o preenchimento dos dados pessoais, a seleção das datas, a confirmação do pagamento simulado e a criação da respetiva reserva.

Foram ainda efetuados testes ao sistema de reservas, permitindo confirmar a criação, consulta, alteração e cancelamento das reservas efetuadas pelos utilizadores. Na área do vendedor, foram testadas todas as operações de gestão de anúncios, incluindo a criação, edição, ativação, pausa e eliminação de anúncios. No painel administrativo, verificou-se o correto funcionamento das funcionalidades de gestão de utilizadores, vendedores, reservas e anúncios.

Durante estes testes foram identificados alguns problemas técnicos relacionados com rotas, validação de formulários, integração da API Amadeus, funcionamento do carrinho de compras e configuração da base de dados. Todos os erros encontrados foram analisados e corrigidos ao longo do desenvolvimento, recorrendo à depuração do código, testes sucessivos e validação das alterações efetuadas.

A realização contínua destes testes permitiu aumentar a estabilidade, a fiabilidade e o desempenho da aplicação, garantindo que todas as funcionalidades principais da plataforma funcionam corretamente e de acordo com os objetivos definidos para o projeto.

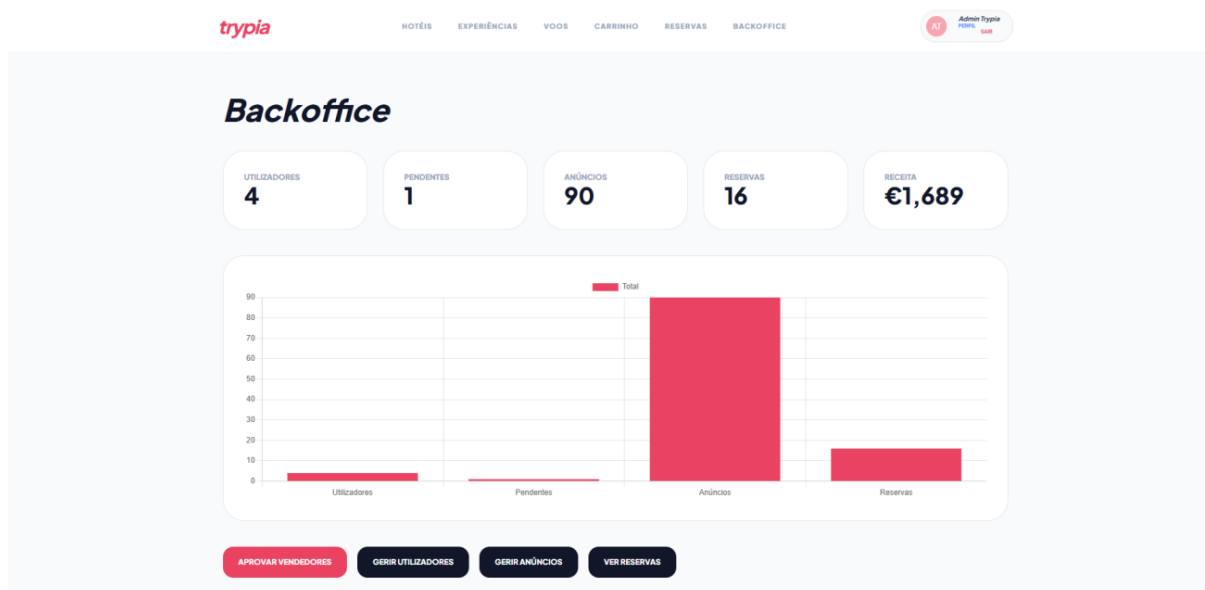


Figura 21: Teste Backoffice

3.18. OTIMIZAÇÃO DA APLICAÇÃO

Durante o desenvolvimento da plataforma Trypia, foram aplicadas diversas técnicas de otimização com o objetivo de tornar a aplicação mais organizada, eficiente e fácil de manter. Desde o início do projeto foi seguida a arquitetura MVC (Model-View-Controller) disponibilizada pelo Laravel, permitindo separar a lógica da aplicação, a gestão dos dados e a interface apresentada ao utilizador. Esta organização facilitou o desenvolvimento de novas funcionalidades e tornou o código mais limpo e reutilizável.

Foi também feita uma separação clara entre o Frontoffice e o Backoffice da aplicação. O Frontoffice reúne todas as funcionalidades destinadas aos utilizadores finais, como a pesquisa de hotéis, experiências e voos, o carrinho de compras e as reservas. O Backoffice concentra as ferramentas de gestão utilizadas pelos vendedores e administradores, permitindo administrar anúncios, utilizadores e reservas de forma independente.

Na construção da interface gráfica foi utilizado o Tailwind CSS, permitindo reutilizar classes utilitárias e reduzir significativamente a quantidade de código CSS personalizado. Esta abordagem contribuiu para uma maior consistência visual em todas as páginas da plataforma e facilitou futuras alterações ao design.

Ao nível da estrutura do projeto, os ficheiros foram organizados segundo a estrutura recomendada pelo Laravel, distribuindo controllers, models, views, rotas e serviços em diretórios próprios. Esta organização tornou o código mais legível e simplificou a manutenção da aplicação.

Foram igualmente efetuadas pequenas otimizações na comunicação com a base de dados, utilizando os relacionamentos definidos nos modelos Eloquent para reduzir código repetido e facilitar a obtenção de informação relacionada entre tabelas. Esta abordagem permitiu tornar as consultas mais simples e melhorar a organização geral da aplicação.

A aplicação beneficia ainda da reutilização de componentes e layouts Blade, evitando duplicação de código em elementos comuns, como a barra de navegação, rodapé e estrutura principal das páginas. Esta reutilização permitiu acelerar o desenvolvimento e garantir uma aparência uniforme em toda a plataforma.

No conjunto, estas otimizações contribuíram para tornar a plataforma Trypia mais organizada, escalável e preparada para futuras evoluções, facilitando tanto o desenvolvimento como a manutenção do projeto.

```

6 <title>@yield('title', 'Trypia')</title>
7 <script src="https://cdn.tailwindcss.com"></script>
8 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
9 <style>
10 @import url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;600;700;800&display=swap');
11 body{font-family:'Plus Jakarta Sans',sans-serif}.hero-bg{background:linear-gradient(to top right, #1a202c 49%, #2d3748 49% 51%, #2d3748 51% 53%, #1a202c 53%);background-size: 400% 400%;background-position: center;animation: gradient 15s ease;position: relative;min-height: 400px;overflow: hidden}
12 </style>
13 </head>
14 <body class="bg-slate-50 text-slate-900">
15 @php($u = \App\Support\CurrentUser::get())
16 <nav class="bg-white/95 backdrop-blur-md sticky top-0 z-50 border-b border-slate-100">
17 <div class="max-w-7xl mx-auto px-6 h-20 flex justify-between items-center">
18 <a href="{{ route('home') }}" class="text-rose-500 text-3xl font-black italic tracking-tighter">trypia</a>
19 <div class="hidden md:flex space-x-10 text-[11px] font-extrabold text-slate-400 uppercase tracking-widest">
20 <a href="{{ route('listings.hotels') }}" class="hover:text-rose-500 transition-all">Hotéis</a>
21 <a href="{{ route('listings.experiences') }}" class="hover:text-rose-500 transition-all">Experiências</a>
22 <a href="{{ route('listings.flights') }}" class="hover:text-rose-500 transition-all">Voos</a>
23 @if($u && $u->isApprovedSeller())<a href="{{ route('cart.index') }}" class="hover:text-rose-500 transition-all">Carrinho @if($u->cartItems()->count())<span class="font-size-10px font-weight-bold text-rose-500">{{ $u->cartItems()->count() }}</span></a>
24 @if($u && $u->isApprovedSeller())<a href="{{ route('seller.listings') }}" class="hover:text-rose-500 transition-all">Os meus</a>
25 @if($u && $u->isAdmin())<a href="{{ route('admin.dashboard') }}" class="hover:text-rose-500 transition-all">Backoffice</a>
26 </div>
27 </div>
28 <div>
29 @if($u)
30 <div class="flex items-center gap-4 bg-slate-50 p-1.5 pr-5 rounded-full border">
31 
32 <div class="flex flex-col">
33 <span class="text-[10px] font-black italic text-slate-700 leading-none">{{ $u->name }}</span>
34 <div class="flex gap-2">
35 <a href="{{ route('profile.edit') }}" class="text-[8px] font-bold text-blue-500 uppercase hover:underline">Editar</a>
36 <a href="{{ route('logout') }}" class="text-[8px] font-bold text-rose-500 uppercase">Logout</a>
37 </div>
38 </div>
39 @else
40 <a href="{{ route('login') }}" class="bg-slate-900 text-white px-8 py-3 rounded-full text-[10px] font-black uppercase">Login</a>
41 @endif
42 </div>
43 </nav>
44
45
46 @if(session('success'))<div class="max-w-7xl mx-auto mt-6 px-6"><div class="bg-emerald-50 border border-emerald-200 text-emerald-700 p-4">
47 @if(session('error'))<div class="max-w-7xl mx-auto mt-6 px-6"><div class="bg-rose-50 border border-rose-200 text-rose-700 p-4 rounded-2xl">
48 @if($errors->any())<div class="max-w-7xl mx-auto mt-6 px-6"><div class="bg-rose-50 border border-rose-200 text-rose-700 p-4 rounded-2xl">
49
50 @yield('content')

```

Figura 22 : Código Layout principal

3.19. SEGURANÇA DA APLICAÇÃO

Durante o desenvolvimento da plataforma Trypia, foram implementadas diversas medidas de segurança com o objetivo de proteger os dados dos utilizadores e garantir que apenas pessoas

autorizadas conseguem aceder às diferentes funcionalidades da aplicação. O Laravel disponibiliza vários mecanismos de segurança integrados, os quais foram utilizados ao longo do desenvolvimento do projeto.

Uma das principais medidas implementadas foi o sistema de autenticação, responsável por controlar o acesso à plataforma. Apenas utilizadores autenticados podem aceder às áreas privadas, como o perfil, histórico de reservas, área do vendedor e painel administrativo. Sempre que um utilizador inicia sessão, a aplicação verifica as suas credenciais antes de permitir o acesso às funcionalidades protegidas.

As palavras-passe dos utilizadores são armazenadas de forma segura através de hashing, utilizando os mecanismos disponibilizados pelo Laravel. Desta forma, as palavras-passe nunca são guardadas em texto simples na base de dados, aumentando significativamente a segurança da informação.

Todos os formulários da aplicação incluem um sistema de validação de dados, garantindo que apenas informações válidas são processadas e armazenadas. Esta validação reduz a possibilidade de erros, protege a base de dados e impede o envio de dados incorretos ou maliciosos.

Para impedir acessos não autorizados às áreas privadas da plataforma, foram implementados mecanismos de controlo de acesso que verificam o tipo de utilizador autenticado antes de permitir determinadas operações. Assim, apenas vendedores podem gerir os seus anúncios e apenas administradores podem aceder ao painel administrativo e às funcionalidades de gestão da plataforma.

A aplicação utiliza ainda o token CSRF (Cross-Site Request Forgery), incluído automaticamente nos formulários através da diretiva @csrf do Laravel. Este mecanismo protege a aplicação contra pedidos fraudulentos provenientes de sites externos, garantindo que apenas pedidos legítimos são processados pelo servidor.

No conjunto, estas medidas contribuíram para aumentar a segurança, a fiabilidade e a proteção dos dados armazenados na plataforma, assegurando um funcionamento seguro para todos os utilizadores.

```

<form method="POST" action="{{ route('admin.listings.update', $listing) }}" class="grid md:grid-cols-2 gap-5">
  @csrf
  @method('PUT')

  <select name="type" class="p-4 rounded-2xl bg-slate-50 font-bold">
    <option value="Hotel" {{ old('type', $listing->type) === 'Hotel' ? 'selected' : '' }}>Hotel</option>
    <option value="Experiência" {{ old('type', $listing->type) === 'Experiência' ? 'selected' : '' }}>Experiência</option>
    <option value="Voo" {{ old('type', $listing->type) === 'Voo' ? 'selected' : '' }}>Voo</option>
  </select>

  <select name="status" class="p-4 rounded-2xl bg-slate-50 font-bold">
    <option value="ativo" {{ old('status', $listing->status) === 'ativo' ? 'selected' : '' }}>Ativo</option>
    <option value="pausado" {{ old('status', $listing->status) === 'pausado' ? 'selected' : '' }}>Pausado</option>
  </select>

  <input name="title" value="{{ old('title', $listing->title) }}" placeholder="Título" class="p-4 rounded-2xl bg-slate-50 font-bold">
  <input name="location" value="{{ old('location', $listing->location) }}" placeholder="Localização / companhia" class="p-4 rounded-2xl bg-slate-50 font-bold">
  <input name="price" value="{{ old('price', $listing->price) }}" placeholder="Preço" class="p-4 rounded-2xl bg-slate-50 font-bold">
  <input name="rating" value="{{ old('rating', $listing->rating ?: 4.8) }}" placeholder="Rating" class="p-4 rounded-2xl bg-slate-50 font-bold">
  <input name="image" value="{{ old('image', $listing->image) }}" placeholder="Imagem principal URL" class="md:col-span-2 p-4 rounded-2xl bg-slate-50 font-bold">
  <textarea name="description" class="md:col-span-2 p-4 rounded-2xl bg-slate-50 font-bold" rows="4" placeholder="Descrição">
  <textarea name="gallery_text" class="md:col-span-2 p-4 rounded-2xl bg-slate-50 font-bold" rows="4" placeholder="Galeria: 1 URL">
  <textarea name="amenities_text" class="md:col-span-2 p-4 rounded-2xl bg-slate-50 font-bold" rows="3" placeholder="Comodidades">

  <button class="md:col-span-2 bg-rose-500 text-white p-4 rounded-2xl font-black uppercase text-xs tracking-widest">Guardar alterações
</form>

```

Figura 23 : Token CSRF

4. PROBLEMAS E SOLUÇÕES ENCONTRADAS

Durante o desenvolvimento do projeto surgiram diversos problemas técnicos que tiveram de ser resolvidos para garantir o correto funcionamento da aplicação. A resolução destas dificuldades permitiu aprofundar os conhecimentos sobre Laravel, bases de dados MySQL, integração de APIs e desenvolvimento web.

Um dos primeiros problemas encontrados foi o erro "Target class does not exist", que surgiu durante a configuração inicial do Laravel. Este erro estava relacionado com dependências em falta e com a configuração do autoloader do projeto. Para o resolver, foi necessário limpar a cache da aplicação, reinstalar as dependências através do Composer e verificar a configuração dos providers.

Outro problema frequente foi o erro "Package discovery failed", provocado por falhas durante a instalação de algumas dependências. A solução passou pela correção do ficheiro composer.json, limpeza da cache do Composer e reinstalação dos pacotes necessários.

Durante o desenvolvimento da base de dados surgiu também o erro "No such table: listings", que indicava a inexistência da tabela utilizada pela aplicação. Este problema foi resolvido através da criação das migrations em falta e da correta configuração da base de dados MySQL no MySQL Workbench.

Foram igualmente encontrados erros do tipo "Route not defined", causados por rotas inexistentes ou nomes incorretos nas views. Após a revisão do ficheiro routes/web.php, as rotas foram corrigidas e as referências existentes nas páginas da aplicação foram atualizadas.

Na integração da API Amadeus verificou-se que, em determinadas situações, eram apresentados resultados antigos em vez dos dados da pesquisa atual. Após analisar o funcionamento da aplicação, foi corrigida a lógica responsável pelo processamento das respostas da API, garantindo que apenas eram apresentados os resultados correspondentes à pesquisa realizada pelo utilizador.

Outro desafio consistiu na implementação do carrinho de compras. Inicialmente, as reservas eram criadas diretamente, sem existir um processo intermédio de confirmação. Para melhorar a experiência do utilizador foi desenvolvido um carrinho que permite adicionar vários serviços antes da confirmação final, seguido de um processo de checkout dividido em várias etapas.

Por fim, foi identificado um problema relacionado com as datas dos voos, uma vez que estas eram alteradas durante o processo de checkout. Este comportamento foi corrigido através da implementação de validações adicionais e da preservação dos dados originais recebidos da API Amadeus, garantindo que a informação apresentada ao utilizador permanecia correta até à conclusão da reserva.

A resolução destes problemas contribuiu significativamente para melhorar a estabilidade, a organização e a fiabilidade da aplicação, permitindo concluir o projeto com um funcionamento consistente e próximo do comportamento esperado numa plataforma real de reservas online.

5. CONCLUSÃO

O projeto Trypia foi concluído com sucesso, tendo cumprido os principais objetivos definidos no início da Prova de Aptidão Profissional. A aplicação desenvolvida permite aos utilizadores pesquisar hotéis, experiências e voos, adicionar serviços ao carrinho de compras e concluir reservas através de um processo de checkout organizado por etapas. Para além destas funcionalidades, a plataforma integra um sistema de autenticação de utilizadores, perfil pessoal, área de vendedor, painel administrativo e comunicação com a API Amadeus para pesquisa de voos.

A realização deste projeto permitiu aplicar, de forma prática, os conhecimentos adquiridos ao longo do Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos, sobretudo nas áreas de programação web, bases de dados, arquitetura MVC, Laravel, PHP, MySQL, Tailwind CSS e integração de APIs. Durante o desenvolvimento foi igualmente necessário recorrer a capacidades de organização, autonomia, investigação e resolução de problemas, competências essenciais para qualquer profissional da área da informática.

Uma das funcionalidades de maior destaque foi a integração da API Amadeus, permitindo incorporar a pesquisa de voos na mesma plataforma utilizada para hotéis e experiências. Esta integração tornou o projeto mais completo e aproximou-o das soluções atualmente utilizadas pelas principais plataformas de reservas online, demonstrando a capacidade de trabalhar com serviços externos e dados em tempo real.

Ao longo do desenvolvimento foram realizados diversos testes para validar todas as funcionalidades da aplicação, permitindo identificar e corrigir erros relacionados com autenticação, gestão de reservas, carrinho de compras, área administrativa, validação de dados e comunicação com a API. Este processo contribuiu para aumentar a estabilidade, a fiabilidade e o desempenho da plataforma.

Embora todos os objetivos inicialmente definidos tenham sido alcançados, existem várias funcionalidades que poderão ser implementadas futuramente para tornar a plataforma ainda mais completa. Entre essas melhorias destacam-se a integração de sistemas de pagamento reais, o envio automático de e-mails de confirmação, a implementação de um sistema de avaliações e comentários, a utilização de mapas interativos, o desenvolvimento de uma aplicação móvel e a integração com novos fornecedores de serviços turísticos.

Para além das competências técnicas adquiridas, este projeto permitiu desenvolver capacidades de planeamento, gestão do tempo, organização do trabalho, análise de requisitos e documentação técnica, contribuindo para uma melhor preparação para futuros desafios académicos e profissionais.

Em conclusão, considero que a realização da Prova de Aptidão Profissional constituiu uma experiência extremamente enriquecedora, permitindo consolidar os conhecimentos adquiridos ao longo do curso e aplicá-los na construção de uma aplicação web moderna, funcional e organizada. O projeto Trypia representa uma etapa importante do meu percurso académico, demonstrando a capacidade de desenvolver soluções informáticas completas através da utilização de tecnologias

atuais e boas práticas de desenvolvimento de software, constituindo uma base sólida para o meu futuro profissional na área da informática.

REFERÊNCIAS BIBLIOGRÁFICAS

Laravel. (2026). Laravel Documentation. <https://laravel.com/docs>

PHP. (2026). PHP Manual. <https://www.php.net/manual>

MySQL. (2026). MySQL Documentation. <https://dev.mysql.com/doc/>

Tailwind CSS. (2026). Tailwind CSS Documentation. <https://tailwindcss.com/docs>

Amadeus for Developers. (2026). Amadeus Self-Service APIs Documentation.
<https://developers.amadeus.com/>

Mozilla Developer Network. (2026). MDN Web Docs. <https://developer.mozilla.org/>

IDENTIFICAÇÃO DO ALUNO

NOME COMPLETO: RUI PEDRO DE SOUSA ZINA

N.º DE PROCESSO: 104722

CICLO DE ESTUDOS: ENSINO SECUNDÁRIO

TELEMÓVEL: 926073596

E-MAIL: RUIPEDROZINA23@GMAIL.COM
